

Catálogo operativo de funcionalidades del Cruzr S2

Fecha de verificación: 14 de agosto de 2026

Unidad: Cruzr S2, `HW_TYPE=cruzr_s2_v1`

Motion: `zs2_motion-v0.26.10`

Este documento enumera las funcionalidades que están disponibles actualmente en esta unidad, con un ejemplo inmediato de uso. Se basa en los contenedores, acciones, servicios y tópicos observados en el robot y en las pruebas físicas ya realizadas.

No es un catálogo comercial de capacidades posibles. Una interfaz declarada en un mensaje ROS 2 no se considera una función operativa hasta confirmar también el servidor, su configuración y un resultado real.

1. Convenciones y acceso

Los ejemplos se clasifican así:

- **LECTURA:** no ordena movimientos.
- **CAMBIO DE ESTADO:** activa o desactiva un proceso, pero no debería mover el robot.
- **MOVIMIENTO:** puede mover base, cabeza, cintura, elevador o brazos.

Para ejecutar los scripts locales:

```
cd /home/lacuna/proyectos/Robots/Humanoide
```

Acceso al PC `vision` por el hotspot del robot:

```
ssh walker@192.168.42.2
```

Con Ethernet también se puede acceder directamente a:

```
ssh walker@192.168.11.3    # vision
ssh walker@192.168.11.2    # motion
```

Shell ROS 2 de diagnóstico en `vision`:

```
docker exec -it walker-ros.ros2-1 bash
source /opt/ros/humble/setup.bash
```

Shell ROSA de navegación en `vision`:

```
docker exec -it walker-nav.nav_taskmanager-1 bash
source /opt/walker/setup.bash
```

Shell ROSA de manipulación en `motion`:

```
docker exec -it walker-motion.manipulation_robot_app-1 bash
source /opt/walker/setup.bash
```

Para inventariar las interfaces activas sin ordenar movimientos:

```
rosa action list
ros2 service list -t
ros2 topic list -t
```

Si el daemon de ROS 2 falla, utilizar `--no-daemon` solamente en los subcomandos que lo admiten, por ejemplo:

```
ros2 topic list --no-daemon -t
ros2 topic info --no-daemon -v /emb/battery_state
ros2 topic echo --no-daemon --once /emb/battery_state
```

No añadir `--no-daemon` a `ros2 topic pub`: esa variante del CLI no lo acepta.

2. Resumen de lo utilizable ahora

Función	Estado actual	¿Mueve el robot?
Estado de baterías, cargador y paros	Probada	No
Cámaras RGB/RGBD, profundidad y estéreo	Activas	No
Nube de puntos estéreo	Activa	No
Lidar delantero y trasero	Activos	No
Detector especializado de contenedor <code>workbin</code>	Probado	No
Estimación de pose 6D de la caja	Probada	No
Detector AprilTag	Servidor activo; prueba física pendiente	No
Profundidad de una ROI estéreo	Servidor activo; prueba física pendiente	No
ASR, reconocimiento de voz	Probado en inglés	No
TTS, síntesis de voz	Probada en inglés	No
Mapas, localización y waypoints	Probados	Sí
Navegación con evitación de obstáculos	Probada	Sí
Movimientos predefinidos de brazos sin manos	Uno probado; otro instalado	Sí
Agarre, elevación y depósito de la caja azul	Probados	Sí
Transporte corto de una caja con la base	Probado por partes/en desarrollo	Sí

Función	Estado actual	¿Mueve el robot?
Ruta interna <code>test_route_01</code>	Navegación probada; ciclo cargado en validación	Sí
Recuperación automática a <code>home</code>	Implementada y usada	Sí

3. Diagnóstico de alimentación y seguridad

3.1 Baterías

LECTURA — dentro de `walker-ros.ros2-1`:

```
timeout 10 ros2 topic echo --no-daemon --once /emb/battery_state
```

El campo `batsoc` es el porcentaje de carga de cada pack. Deben revisarse ambos; no se debe usar la media para ocultar una diferencia importante entre packs.

Lectura compacta:

```
timeout 10 ros2 topic echo --no-daemon --once /emb/battery_state |
grep -E 'charge_status|voltage:|current:|temperature:|batsoc:|sn:'
```

3.2 Cargador y paros

LECTURA:

```
timeout 10 ros2 topic echo --no-daemon --once /emb/chrg_input_status
timeout 10 ros2 topic echo --no-daemon --once /emb/estop_key_state
timeout 10 ros2 topic echo --no-daemon --once /emb/servo_estop_key_state
```

Los scripts de movimiento bloquean la ejecución si detectan cargador conectado o un paro activo. La ausencia de fallo ROS no sustituye al paro físico preparado.

4. Cámaras, profundidad y nube de puntos

Están activos los controladores de cámara de chasis, cintura, estéreo, RGB y fisheye. Las fuentes más relevantes son:

- cámara RGBD frontal del chasis;
- cámara RGBD frontal de la cintura;
- cámara estéreo de la cabeza;
- profundidad estéreo y nube de puntos;
- cámaras fisheye usadas por navegación/localización.

4.1 Comprobar que una imagen RGB llega

LECTURA — dentro de `walker-ros.ros2-1`:

```
timeout 8 ros2 topic hz /sensor/camera/stereo/color/raw
```

`timeout` terminará el comando automáticamente; el código 124 sólo significa que agotó esos ocho segundos. Debe mostrarse una frecuencia mayor que cero.

Para localizar los nombres exactos de las demás cámaras de esta versión:

```
ros2 topic list --no-daemon -t | grep -E '/sensor/camera/.*(color|rgb|image)'
```

4.2 Comprobar profundidad

LECTURA:

```
timeout 8 ros2 topic hz /sensor/camera/stereo/depth/raw
```

4.3 Comprobar la nube de puntos

LECTURA:

```
timeout 8 ros2 topic hz /sensor/camera/stereo/pointcloud/raw  
ros2 topic info --no-daemon -v /sensor/camera/stereo/pointcloud/raw
```

Esto confirma el flujo; no guarda un fichero PCD. Para una aplicación propia se debe suscribir al tópico `sensor_msgs/msg/PointCloud2` y conservar también su `frame_id` y sello temporal.

4.4 Consultar la calibración de cámara

LECTURA:

```
ros2 topic list --no-daemon -t | grep -E '/sensor/camera/.*(info|camera_info)'
```

Después se consulta el tópico encontrado:

```
timeout 10 ros2 topic echo --no-daemon --once NOMBRE_DEL_TOPICO
```

No se deben copiar intrínsecos de otra unidad; hay que usar los publicados por este robot.

5. Lidar y percepción de obstáculos

Los lidar delantero y trasero están activos y alimentan la navegación.

5.1 Verificar ambos lidar

LECTURA:

```
timeout 8 ros2 topic hz /sensor/lidar/front  
timeout 8 ros2 topic hz /sensor/lidar/back  
ros2 topic info --no-daemon -v /sensor/lidar/front  
ros2 topic info --no-daemon -v /sensor/lidar/back
```

5.2 Evitación de obstáculos

No se inicia con una orden independiente: forma parte del stack de navegación. Ya se observó que el robot modifica el recorrido ante obstáculos durante una navegación a waypoint.

Ejemplo operativo: navegar a un punto con el procedimiento de la sección 10 y colocar únicamente un obstáculo ligero y visible, bajo supervisión y con paro preparado. No utilizar personas para probar el frenado.

6. Detector especializado de caja azul `workbin`

Ésta es la función de percepción de objetos confirmada. No es un detector general: está configurado para un contenedor tipo BYD de aproximadamente `0.60 × 0.40 × 0.22 m` y devuelve el objeto `workbin` y su pose.

El contenedor debe estar visible, estable, con el lado largo aproximadamente paralelo a los hombros y sin oclusiones importantes.

6.1 Medición rápida sin mover el robot

LECTURA — desde el PC Ubuntu:

```
cd /home/lacuna/proyectos/Robots/Humanoide
./scripts/cruze_blue_workbin_cycle.sh --measure-box-fast
```

Salida esperada:

```
BOX_POSE_CAMERA=x y z yaw
```

`x` representa el error lateral en la cámara, `z` la distancia aproximada y `yaw` el giro de la caja. Si no aparece `workbin`, el script rechaza la medida.

6.2 Llamada directa al detector

LECTURA — dentro de `walker-motion.manipulation_robot_app-1` en `motion`:

```
rosa action send_goal /cv/task/transport_action \
  cv_task_msgs/action/VisionActionTask \
  '{"task_type":"transport","trans_inputs":{"camera_name":"head","task_stage":"grasp","box_size":
{"x":0.60,"y":0.40,"z":0.22}}}'
```

Un resultado válido debe terminar con `status=4`, `ok: True`, `object_name: workbin` y al menos una pose en `box_pose`.

6.3 Preparar únicamente la visión

MOVIMIENTO DE CABEZA:

```
./scripts/cruze_blue_workbin_cycle.sh --prepare-vision
```

Esta orden baja la cabeza; no mueve brazos ni chasis.

7. Detector AprilTag

El contenedor `walker-drivers.apriltag_detector-1` y los servicios de inicio están activos. Hace falta una etiqueta física, conocer su ID y medir su lado impreso en metros.

7.1 Detectar una etiqueta

CAMBIO DE ESTADO, sin movimiento — dentro de `walker-ros.ros2-1`:

```
ros2 service call /apriltag/start_detecting \
  sensor_task_msgs/srv/AprilTagStartDetecting \
  "{start_detecting: true, img_topic_name: '/sensor/camera/stereo/color/raw', tag_id: 0, tag_size: 0.16,
  tag_frame: 'apriltag_0'}"
```

En otro terminal ROS 2:

```
ros2 topic echo --no-daemon /sensor/camera/stereo/april_tag/results
```

El ejemplo supone ID 0 y lado impreso de 0,16 m. Hay que sustituir ambos datos por los de la etiqueta real; un tamaño incorrecto produce una distancia 3D incorrecta.

7.2 Detener la detección

CAMBIO DE ESTADO:

```
ros2 service call /apriltag/start_detecting \
  sensor_task_msgs/srv/AprilTagStartDetecting \
  "{start_detecting: false, img_topic_name: '/sensor/camera/stereo/color/raw', tag_id: 0, tag_size: 0.16,
  tag_frame: 'apriltag_0'}"
```

La detección de bundles también está instalada mediante `/apriltag/start_bundle_detecting`, pero requiere un YAML de bundle compatible; no se proporciona una orden de ejecución hasta definir ese fichero.

8. Profundidad estéreo de una región de interés

La acción `/cv/task/stereo_depth_roi_action` está activa. Permite pedir RGB y profundidad sólo para un rectángulo de la imagen.

8.1 Verificar resolución antes de elegir la ROI

LECTURA:

```
ros2 topic list --no-daemon -t | grep '/sensor/camera/stereo/.info'
timeout 10 ros2 topic echo --no-daemon --once TOPICO_CAMERA_INFO
```

8.2 Solicitar una ROI de prueba

LECTURA — dentro de un contenedor con `/opt/walker/setup.bash` y `rosa`:

```
rosa action send_goal /cv/task/stereo_depth_roi_action \  
cv_task_msgs/action/StereoDepthRoi \  
'{"input_roi":{"x_offset":100,"y_offset":100,"height":100,"width":100,"do_rectify":false}}'
```

La ROI debe quedar dentro de la resolución publicada. Un resultado correcto contiene `success: true`, la ROI confirmada y los mapas RGB/profundidad.

9. Voz

9.1 Hacer que el robot hable en inglés

AUDIO, sin movimiento — desde el PC Ubuntu y con acceso directo a `192.168.11.3`:

```
./scripts/cruzeir_say_english.sh --check  
./scripts/cruzeir_say_english.sh "Welcome. The system is ready."
```

`--check` comprueba la acción TTS sin reproducir audio.

Para listar las voces instaladas, dentro de `walker-ros.ros2-1`:

```
ros2 service call /sys/speech/get_speaker_list \  
sys_task_msgs/srv/GetSpeakerList "{}"
```

9.2 Reconocer voz en inglés

CAMBIO DE ESTADO, sin movimiento — dentro de `walker-ros.ros2-1`:

```
ros2 service call /sys/asr/enable std_srvs/srv/SetBool "{data: true}"  
ros2 service call /sys/run_record/enable std_srvs/srv/SetBool "{data: true}"  
ros2 topic echo --no-daemon /sys/speech/asr
```

Al hablar se espera una salida como:

```
text: 'please wave. '  
language: <|en|>
```

Para terminar la captura:

```
ros2 service call /sys/run_record/enable std_srvs/srv/SetBool "{data: false}"
```

`run_record` activa el flujo de micrófono/ASR; no implica que se cree y conserve un archivo de audio. El ASR puede clasificar erróneamente una frase inglesa como china, por lo que una orden de movimiento debe exigir coincidencia explícita y timeout.

10. Mapas, localización y navegación autónoma

El mapa confirmado es `test_route_01`, con los puntos:

```
START, PAS01, PAS02, PASO 3, PAS04, FINISH
```

La navegación usa localización, lidar, mapas de coste y evitación de obstáculos.

10.1 Comprobar mapa y localización sin mover

LECTURA — desde el PC Ubuntu:

```
./scripts/cruze_blue_workbin_map_route_short.sh --check --fast
```

Dentro de `walker-nav.nav_taskmanager-1` también se puede consultar el mapa:

```
rosa action send_goal /vnav/task/command \
  unav_task_msgs/action/Task \
  '{"command":"get_map_name","arg_json":{"{}}"}
```

La pose actual se observa dentro de `walker-ros.ros2-1`:

```
timeout 10 ros2 topic echo --no-daemon --once /nav/robot_pose
```

10.2 Navegar a un waypoint

MOVIMIENTO DE BASE — dentro de `walker-nav.nav_taskmanager-1`:

```
rosa action send_goal /vnav/task/command \
  unav_task_msgs/action/Task \
  '{"command":"navigation_start","arg_json":{"target_point\
  {"map_name\":"test_route_01\","mode\":"logo_nav\","id\":"PAS01\"},"'}
```

Antes de ejecutarlo: cargador y Ethernet desconectados, robot correctamente localizado, pasillo despejado para toda su anchura y paro preparado.

10.3 Parar una navegación por software

En otro terminal del mismo contenedor:

```
rosa action send_goal /vnav/task/command \
  unav_task_msgs/action/Task \
  '{"command":"navigation_stop","arg_json":{"{}}"}
```

Ctrl+C sólo puede cortar el cliente; no debe suponerse que cancela la misión interna. Para una emergencia se utiliza el paro físico, no esta acción ROSA.

10.4 Crear o editar mapas y puntos

La interfaz web ya se ha utilizado en:

```
http://192.168.11.3/map/navigation
```


Permite crear el mapa con lidar, localizar el robot y crear waypoints. Para una segunda mesa conviene crear un waypoint de aproximación despejado delante de la mesa, no un punto debajo ni pegado a ella. La aproximación final a la caja o al apoyo debe hacerse con percepción/odometría local.

11. Movimientos predefinidos de brazos sin manos

La lista blanca actual contiene:

- `fist_up_s2`: brazo derecho; probado físicamente en esta unidad.
- `cruze/wave_arm`: saludo de brazo derecho y retorno a cero; tarea oficial instalada, pendiente de primera validación física controlada.

11.1 Listar y comprobar

LECTURA:

```
./scripts/cruze_brazos_sin_manos.sh --list
./scripts/cruze_brazos_sin_manos.sh --check
```

11.2 Ejecutar el movimiento ya probado

MOVIMIENTO DE BRAZO:

```
./scripts/cruze_brazos_sin_manos.sh --run fist_up_s2
```

11.3 Ejecutar el saludo oficial

MOVIMIENTO DE BRAZO:

```
./scripts/cruze_brazos_sin_manos.sh --run cruze/wave_arm
```

Se exige zona completa de brazos despejada, robot estable, cargador desconectado y paro preparado. No usar `--run-all` como primera prueba de una rutina nueva.

12. Agarre, elevación y depósito del contenedor azul

El flujo determinista validado combina el detector `workbin` con tareas de manipulación. No utiliza GR00T/VLA.

12.1 Comprobar el sistema

LECTURA:

```
./scripts/cruze_blue_workbin_cycle.sh --check
```

12.2 Ciclo sin mover la base

MOVIMIENTO DE CABEZA Y BRAZOS:

```
./scripts/cruze_blue_workbin_cycle.sh --run --hold 3 --yes --fast
```

Secuencia: baja cabeza, prepara brazos, detecta, sujeta, eleva, mantiene tres segundos, baja hasta contacto y abre. No mueve el chasis y no vuelve a **home** porque la mesa sigue dentro de la trayectoria de los brazos.

12.3 Agarrar y dejar la caja suspendida

MOVIMIENTO DE BRAZOS:

```
./scripts/cruze_blue_workbin_cycle.sh --grasp --yes
```

12.4 Depositar una caja ya sujeta

MOVIMIENTO DE BRAZOS:

```
./scripts/cruze_blue_workbin_cycle.sh --deposit-held --yes
```

Estas dos órdenes sólo deben usarse si el registro del script corresponde al agarre vigente y nadie ha modificado físicamente caja, mesa o brazos.

13. Aproximación visual y transporte corto de la caja

13.1 Centrar el robot sin agarrar

MOVIMIENTO DE CABEZA Y BASE:

```
./scripts/cruze_blue_workbin_carry_back.sh --align-only --yes --fast
```

La función mide centro, profundidad y orientación, corrige el chasis y termina antes de mover los brazos.

13.2 Agarrar después de centrarse

MOVIMIENTO DE BASE, CABEZA Y BRAZOS:

```
./scripts/cruze_blue_workbin_carry_back.sh --grasp-only --yes --fast
```

13.3 Ciclo corto completo

MOVIMIENTO DE BASE, CABEZA Y BRAZOS:

```
./scripts/cruze_blue_workbin_carry_back.sh --run --yes --fast
```

Secuencia prevista: centra, agarra, retrocede 0,50 m, vuelve a la pose de agarre, deposita, se separa de la mesa y termina en **home**.

Este ciclo integra funciones que se han validado por partes y continúa en ajuste. Debe permanecer supervisado; no equivale todavía a una misión industrial homologada.

14. Recorridos del mapa transportando la caja

14.1 Ruta corta actual

MOVIMIENTO COMPLETO:

```
./scripts/cruZR_blue_workbin_map_route_short.sh --run --yes --fast
```

Ruta configurada:

```
caja -> START -> PAS01 -> mesa -> depósito -> home
```

Desde **PAS01** vuelve directamente a la pose de aproximación de la mesa.

14.2 Ruta larga

MOVIMIENTO COMPLETO:

```
./scripts/cruZR_blue_workbin_map_route.sh --run --yes --fast
```

Ruta configurada:

```
START -> PAS01 -> PAS02 -> PASO 3 -> PAS04 -> FINISH  
-> PAS04 -> PASO 3 -> PAS02 -> PAS01 -> START  
-> mesa -> depósito -> home
```

La ruta larga requiere pasillos y giros despejados para la anchura total de la caja, supervisión continua y batería suficiente en ambos packs. Sigue siendo una prueba de integración, no una función de producción desatendida.

15. Recuperación automática a **home**

15.1 Diagnóstico sin movimiento

LECTURA:

```
./scripts/cruZR_recover_to_home.sh --check
```

15.2 Recuperar postura

MOVIMIENTO DE BASE Y CUERPO:

```
./scripts/cruZR_recover_to_home.sh --run --yes --fast
```

Si detecta brazos junto a la mesa, retrocede hasta 0,50 m antes de ejecutar la tarea oficial **cruZR/home**. Nunca avanza hacia la mesa. Debe haber al menos 1,50 m libres detrás del robot.

16. Interfaces presentes que aún no deben anunciarse como funciones operativas

Las siguientes señales existen, pero todavía no hay una prueba completa que permita presentarlas como funcionalidades listas para usar:

Interfaz o módulo	Evidencia presente	Qué falta
Detector de caja con cámara de cintura	<code>/cv/waist_front/task/transport_action</code> tiene servidor	Confirmar <code>camera_name</code> , frames y resultado con caja real
Guardado de imagen estéreo	<code>/cv/task/stereo_image_save_action</code> tiene servidor	Contrato validado de <code>task_type</code> , ruta y permisos
Tracking pose 6D	servicios <code>/cv/task/pose_6d_track_request</code> y variante de cintura	Valores oficiales de <code>track_type</code> y prueba de estabilidad
Imagen semántica de navegación	<code>/vnav/perception/chassis_front_rgbd/semantic_image</code>	Confirmar clases, codificación y publicación durante misión
OCR 3D	<code>/vnav/perception/ocr_3d_markers</code>	Confirmar modelo, idiomas, activación y resultado real
Reconocimiento de persona/cara	Campos en mensajes genéricos de visión	No se confirmó servidor/modelo específico ni salida anotada
QR y SPS	Campos en <code>VisionActionTask</code>	No se confirmó configuración, modelo ni valores de <code>task_type</code>
Bundle AprilTag	Servicio activo	Crear y validar YAML de bundle
Recarga autónoma	Acción <code>/vnav/action/recharge</code> activa	Base compatible, mapa, tolerancias e interlocks validados
Comandos LLM	Acciones <code>/sys/speech/llm</code> presentes	Servicio conversacional fiable e integración segura con tareas
Aplicación móvil	No confirmada	APK oficial, compatibilidad y credenciales de UBTECH

Se puede comprobar la existencia de una interfaz sin ejecutarla:

```
rosa action info NOMBRE_DE_LA_ACCION
rosa action type NOMBRE_DE_LA_ACCION
ros2 service type NOMBRE_DEL_SERVICIO
ros2 interface show TIPO_DEL_SERVICIO
ros2 topic info --no-daemon -v NOMBRE_DEL_TOPICO
```

17. VLA/GR00T: disponible como paquete, no operativo ahora

El paquete compartido contiene dataset, código, imágenes Docker y el checkpoint `checkpoint-40000` de GR00T N1.5. Sin embargo, en el robot no aparecen actualmente los contenedores VLA ni las interfaces `/gr00t/trigger_inference` y `/vla_inference_result` activas.

Por tanto:

- el reconocimiento y agarre actuales de la caja no los realiza GR00T;
- los realiza el detector especializado `workbin` más tareas deterministas de visión, manipulación y navegación;
- no existe todavía una orden VLA que pueda incluirse honestamente como ejemplo “usable ahora”;
- instalar el VLA requiere una validación separada de versiones, CUDA, límites articulares, parada, colisión y comportamiento sin publicar movimiento.

18. Regla práctica para nuevas funciones

Una nueva capacidad se añadirá a la parte operativa del catálogo sólo después de superar estas cuatro comprobaciones:

1. El contenedor correspondiente está activo.
2. La acción, servicio o tópico tiene servidor/publicador real.
3. Una llamada sin movimiento devuelve datos coherentes y con frames conocidos.
4. Si mueve el robot, supera una prueba física supervisada y dispone de una recuperación segura.

Esta regla evita confundir “el mensaje contiene un campo” con “el robot ya sabe hacer la tarea”.